MCP로 구성하는 AI 기반 모바일 앱 보안 감사

원본형 · 개념 · 도구 적용 · 감사 시스템 · Skill · 운영 통제

모바일 앱 보안 감사를 MCP와 Skill 중심으로 구조화하는 상세 Slidev 자료

작성일: 2026-04-24 · 범위: Android 앱 감사, MCP Server, Skill 설계, 보안 통제

1. 발표 목표와 기본 지식

MCP로 구성하는 AI 기반 모바일 앱 보안 감사

- 주제: AI 기반 모바일 앱 감사 환경 구성
- 범위: Android 앱 보안 감사 중심
- 핵심 구성: MCP Client, MCP Server, Skill, Evidence, Report
- 목표: 반복 가능한 감사 워크플로우와 증거 기반 보고 체계 수립

2. 발표 목표와 범위

- MCP, Tool, Skill의 차이를 명확히 이해
- 모바일 앱 감사용 MCP 도구체인 구성
- 각 도구의 적용 위치와 사용 예시 파악
- AI 기반 감사 시스템 구조 설계
- 보안 통제와 Human-in-the-loop 운영 원칙 정리

제외 범위

- 무단 대상 공격 자동화
- 무승인 능동 스캔
- 우회 기법의 무분별한 자동 실행

3. 기본 지식: 모바일 앱 보안 감사의 작업 구조

1. 감사 범위 확인
2. APK, IPA, 소스 코드 확보
3. Manifest, 권한, 컴포넌트 분석
4. 취약 패턴과 민감정보 분석
5. 디바이스 실행 후 로그와 화면 수집
6. 런타임 값과 함수 호출 관찰
7. 네이티브 라이브러리 분석
8. WebView와 API 트래픽 분석
9. 증거 정리와 보고서 작성

4. 기본 지식: AI Agent란 무엇인가

구분	특징
일반 LLM	생성과 요약에 강하지만 도구 실행은 못함
AI Agent	목표 분해, 도구 선택, 결과 해석, 다음 행동 결정

- 명령어 변환기
- 분석 보조자
- 증거 관리자
- 보고서 작성자

5. 기본 지식: Tool Calling의 동작 방식

1. Tool 정의 등록
2. 사용자 요청 해석
3. 실행할 Tool 선택
4. 파라미터 생성
5. Tool 실행
6. 결과 수신
7. 보고서나 요약으로 통합

```
{
  "tool": "get_device_info",
  "arguments": {"device_id": "R5CR123ABC4"}
}
```

6. 기본 지식: MCP의 구조

- Host
- MCP Client
- MCP Server
- Tools
- Resources
- Prompts

MCP는 AI 애플리케이션과 외부 도구를 연결하는 표준 프로토콜이다.

7. 기본 지식: MCP와 Tool을 혼용하지 않기

구분	의미
MCP	연결 방식
MCP Server	도구 묶음
Tool	실행 단위
Skill	절차와 템플릿

- 부정확: MCP로 스크린샷을 찍는다
- 정확: Android MCP Server의 스크린샷 Tool을 호출한다

8. 기본 지식: Skill은 무엇을 담당하는가

- 반복 감사 절차 표준화
- 도구 호출 순서 정의
- 판단 기준 제공
- 보고서 템플릿 제공
- 산출물 저장 규칙 정의

```
mobile-app-security-audit/
  SKILL.md
  references/
  templates/
  scripts/
```

9. 기본 지식: 일반 CLI Agent의 기본 도구

- 파일 읽기, 검색, 편집
- 셸 실행
- 패치 적용
- Git 확인
- 웹 조회
- 작업 관리

MCP는 여기에 Android, JADX, Frida, Ghidra, Burp 같은 전문 도구를 추가하는 확장 방식이다.

10. 기본 지식: 감사 기준은 MASVS와 MASTG로 정렬한다

- Finding을 MASVS 항목에 매핑
- 테스트 절차를 MASTG 기반으로 구성
- 도구별 결과를 통제 항목별 증거로 정리

```
Tool Result -> Evidence -> MASVS Mapping -> Finding
```

11. 구축: 테스트 랩 구성

- Python 3.10+
- Node.js LTS
- Git
- Android SDK Platform-Tools
- JDK
- Android 디바이스 또는 에뮬레이터
- MCP 지원 Client
- JADX, Frida, Ghidra, Chrome, Burp, MobSF, Semgrep, CodeQL

12. 구축: MCP Client 설정 구조

```
{
  "mcpServers": {
    "android": {
      "command": "uv",
      "args": ["--directory", "<tool-root>", "run", "server.py"]
    }
  }
}
```

- 서버별 로그와 버전 기록
- 위험 Tool 자동 승인 금지

13. 구축: 전체 감사 도구체인 아키텍처

- 사용자
- AI Agent Client
- Skill
- MCP Server Layer
- 실제 보안 도구
- 대상 앱, 디바이스, API
- Evidence Store
- Report

14. 구축: 승인 범위와 Human-in-the-loop

승인 필요 작업

- 앱 설치와 삭제
- 파일 pull, push
- Frida 스크립트 주입
- 능동 스캔
- 이슈와 PR 생성

자동 실행 가능 작업

- 정보 조회
- 화면 캡처
- 로그 필터링
- 보고서 초안 생성

15. Android MCP Server: 디바이스 제어와 증거 수집

- 디바이스 정보 확인
- 앱 설치, 실행 상태 확인
- 스크린샷 수집
- UI Layout XML 수집
- Logcat 수집
- 패키지 정보와 권한 확인

16. Android MCP Server: 적용 방법

```
adb devices
adb shell getprop ro.product.model
adb shell pm list packages
```

- ADB 연결 안정화
- 대상 디바이스 지정
- MCP Client 설정

17. Android MCP Server: 사용 예시

- 현재 앱 화면 캡처
- UI Layout 분석
- 입력 필드와 버튼 정리
- Login 버튼 좌표 계산
- 이후 인증 관련 logcat 수집

18. JADX MCP: APK 정적 분석

- 디컴파일 코드 분석
- Manifest 분석
- 권한과 컴포넌트 확인
- Exported Component 탐지
- Secret, API Key, Token 탐지
- 저장소, 인증, 네트워크 코드 위치 식별

19. JADX MCP: 적용 방법

- JADX 설치
- MCP 플러그인 설치
- APK 로드
- MCP 서버 실행
- Client에 서버 등록
- 현재 클래스와 Manifest 조회

20. JADX MCP: 사용 예시

- MainActivity 요약
- 하드코딩 API Key, Token, Password 확인
- 관련 코드 라인 정리
- WebView 사용 클래스 검색
- addJavascriptInterface와 setJavaScriptEnabled 확인

21. Frida MCP: 런타임 동적 분석

- 복호화되는 값 확인
- 함수 호출 인자와 반환값 로깅
- 인증, 암호화, 네트워크 흐름 추적
- 동적 클래스 로딩 탐지
- 네이티브 함수 호출 관찰

22. Frida MCP: 적용 방법

```
pip install frida-tools
frida-ps -U
```

- frida-server 준비
- 버전 일치 확인
- spawn 또는 attach 선택
- 테스트 계정 사용

23. Frida MCP: 사용 예시

- validateUser 메서드 호출 추적
- 인자와 반환값 로그
- 실패와 성공 조건 요약
- Cipher.init 호출에서 알고리즘과 모드만 요약

24. Ghidra MCP: 네이티브 코드 분석

- JNI 함수 분석
- 네이티브 코드 디컴파일
- 암호화, 복호화 루틴 식별
- 문자열 복호화 추적
- 안티디버깅, 안티탬퍼링 탐지

25. Ghidra MCP: 적용 방법

- Ghidra 설치
- JDK 설치
- `.so` 추출 후 로드
- Auto Analysis 실행
- MCP 플러그인과 Bridge 연결
- Client에 ghidra 서버 등록

26. Ghidra MCP: 사용 예시

- JNI 함수 검색
- 디컴파일 코드 조회
- 호출 함수와 참조 데이터 추적
- 반환값 생성 방식 설명
- 안티디버깅 문자열 후보 탐색

27. Chrome DevTools / Playwright MCP: WebView와 웹 분석

- DOM 구조 분석
- 로그인과 회원가입 플로우 자동화
- 네트워크 요청과 응답 분석
- LocalStorage, SessionStorage, Cookie 확인
- XSS와 CSP 후보 탐지

28. Chrome DevTools / Playwright MCP: 적용 방법

```
{
  "mcpServers": {
    "chrome-devtools": {"command": "npx", "args": ["-y", "chrome-devtools-mcp@latest"]},
    "playwright": {"command": "npx", "args": ["-y", "@playwright/mcp@latest"]}
  }
}
```

- WebView 디버깅 활성화 확인
- 페이지 콘텐츠를 Agent 지시문으로 신뢰하지 않기

29. Chrome DevTools / Playwright MCP: 사용 예시

- 테스트 로그인 페이지 이동
- 테스트 계정 입력과 로그인 실행
- Network 요청 목록 조회
- Storage와 Cookie 확인
- CSP, Console Error, Mixed Content 점검

30. MobSF / Semgrep / CodeQL: 자동 스캐닝과 정적 분석 확장

- MobSF: 모바일 앱 초벌 분석
- Semgrep: 룰 기반 SAST
- CodeQL: 데이터플로우 기반 정적 분석

자동 스캐닝은 Finding 확정 도구가 아니라 후보 생성 도구다.

31. MobSF / Semgrep / CodeQL: 적용 방법

- MCP Server 직접 연결
- CLI Tool 래핑
- Skill 안의 스크립트화

```
scripts/
  run-mobsf-static-scan.ps1
  run-semgrep-mobile-rules.ps1
  normalize-codeql-results.py
```

32. MobSF / Semgrep / CodeQL: 사용 예시

- APK에 대해 MobSF 정적 분석 실행
- High, Medium 이슈 필터링
- MASVS 기준 분류
- JADX로 추가 확인할 코드 위치 제안

33. Burp / ZAP / GitHub MCP: 네트워크와 운영 워크플로우 확장

- Burp와 ZAP으로 HTTP, API 보안 분석 연결
- GitHub MCP로 이슈와 PR 흐름 연결
- 트래픽 증거와 코드 증거를 같은 보고서에 묶기

34. Burp / ZAP / GitHub MCP: 적용 방법

- 프록시 구성
- 대상 범위 제한
- 자동 승인 정책 신중 설정
- 최소 권한 토큰 사용
- 능동 스캔은 승인 후 수행

35. Burp / ZAP / GitHub MCP: 사용 예시

- `/api/login` 이후 요청 분석
- Authorization 헤더 방식 확인
- 토큰 저장 위치 확인
- 민감정보 평문 전송 여부 확인
- 확정 Finding을 GitHub Issue 초안으로 변환

36. 감사 시스템: AI Mobile Audit Orchestrator 개요

입력값

- APK
- 소스 코드 저장소
- 테스트 디바이스
- 테스트 계정
- API base URL
- 감사 범위 문서

출력값

- Evidence Bundle
- Finding 후보 목록
- MASVS 매핑 표
- 보고서 초안

37. 감사 시스템: 컴포넌트 구성

- AI Agent
- Audit Skill
- Tool Registry
- Approval Gate
- MCP Server Layer
- Evidence Store
- Report Generator
- Policy Guardrail

38. 감사 시스템: 단계별 실행 흐름

1. 대상 등록
2. 기본 정보 수집
3. 정적 분석
4. 런타임 분석
5. 네이티브 분석
6. 웹/API 분석
7. 보고서화

원칙: 먼저 자동 수집, 이후 사람이 검토

39. 감사 시스템: 로그인 기능 감사 시나리오

- 화면 캡처와 UI Layout 수집
- 로그인 관련 코드 분석
- 인증 함수 호출 흐름 관찰
- HTTP 요청 확인
- 토큰 저장 위치 확인
- MASVS 기준으로 매핑

40. 감사 시스템: Finding 생성 예시

```
# Finding: 제목
## 요약
## 영향도
## 영향 범위
## 재현 절차
## 증거
## 기준 매핑
## 권고 조치
## 검증 방법
```

41. Skill 설계: Mobile App Security Audit Skill

- 절차 표준화
- MASVS 기준 매핑
- Finding 품질 균일화
- 증거 누락 방지
- 반복 감사 시간 단축

42. Skill 설계: 감사 단계별 Tool 매핑

- 대상 식별: Android MCP
- Manifest 분석: JADX, MobSF
- Secret 탐지: JADX, Semgrep
- 인증 분석: JADX, Frida, Burp
- 네트워크 분석: Burp, Chrome DevTools
- 네이티브 분석: Ghidra, Frida
- 보고서화: Skill Template

43. Skill 설계: 감사 프롬프트 템플릿

- 대상 등록 프롬프트
- 정적 분석 프롬프트
- 동적 분석 프롬프트
- 보고서화 프롬프트

프롬프트는 짧고 명확하게, 판단 기준은 reference 파일로 분리한다.

44. Skill 설계: Evidence Bundle 표준화

```
evidence/{app_name}/{date}/
  00_scope/
  01_device/
  02_static/
  03_dynamic/
  04_native/
  05_network/
  06_findings/
```

- 증거 경로
- 테스트 일시
- 도구 버전
- Tool Call 요약
- 민감정보 마스킹 여부

45. 운영 통제: MCP 도구 사용 리스크

- 과도한 Tool 권한
- 셸 명령 오남용
- 민감정보 유입
- 프롬프트 인젝션
- 검증되지 않은 MCP Server
- 자동 승인으로 인한 예기치 않은 실행
- 범위 초과 스캔

46. 운영 통제: 안전한 감사 운영 체크리스트

사전

- 승인된 범위 문서 확보
- 테스트 계정 준비
- MCP Server 출처와 버전 확인
- 위험 Tool 자동 승인 비활성화

실행 중

- 능동 스캔 전 승인 확인
- 대상 함수와 경로 확인
- 민감정보 포함 여부 점검

사후

- 증거 무결성 확인
- Finding 후보와 확정 Finding 분리
- 재검증 절차 포함

47. 결론: MCP는 연결, Tool은 실행, Skill은 절차

- MCP는 도구를 연결하는 프로토콜
- Tool은 실제 실행 단위
- Skill은 절차와 판단 기준
- Agent는 보안 도구를 조율하는 오케스트레이터
- 판단과 책임은 감사자에게 남는다

48. Q&A

- 도구 설치 이슈
- MCP 보안 통제
- 적절한 도구 조합
- Skill 운영 방식
- AI가 만든 Finding 검증 절차
- MASVS, MASTG 기준 매핑 방식

49. 참고 자료

- MCP Specification
- Claude Code Tools, MCP, Skills
- OpenAI Codex MCP, Config
- OWASP MASVS, MASTG
- Android MCP, JADX MCP, Frida MCP, GhidraMCP
- Chrome DevTools MCP, Playwright MCP
- Semgrep MCP, CodeQL Development MCP Server
- MobSF
- GitHub MCP Server